

E-Commerce Regional Demand Forecasting

Colin Seiler

Aidan Collins

Aditya Kanojiya

3NF Justification Report

Our schema has 8 tables that we use to model the Olist dataset: Zips, Customers, Sellers, Orders, OrderItems, Products, Payments and Reviews. Everything is put under a commerce schema. BIGINT IDENTITY is used for primary keys instead of the ones provided because they are 32 characters long and would overall slow down the how process. The original IDs are now stored in the Reference cols., so if we need to access them later we can.

To get 1NF every single column needs to store one value. When you have data that has multiple items, like multiple payments or multiple items per order, we now have separated them in Orderitems and Payments instead of trying to fit them all into one field. Lat and lng are also separate in Zips. To get 2NF Non-key columns need to depend on the whole primary key. For the tables that have one PK column this is not an issue. OrderItems has a composite key; OrderItemID, and OrderID. The freight and price and all other components in that table are only about that specific line item and in that specific order, so in the end there are no partial dependencies. Payments works in the same way but with; OrderID, and PaySeq. Hence, the payment information is describing that particular payment, not just the order. To get 3NF you must not have columns that depend on other non-key columns. For our specific scenario Geographic items were the main concern. Since City and state depend on zipcode, a new table Zips was created and we reference it from Orders and Sellers. This prevents us from duplicating city/state everywhere. We did end up leaving Category in the Products table directly. A separate categories table could've made more normalization but we didn't really need it since we were not going to be storing the english translations into the database. With this said, all of the order timestamps are independent of one another, the approval time does not determine delivery time or anything of that nature, which leaves us with no transitive dependencies there.

Products and orders are many-to-many. Meaning that an order can have multiple products, and a product can appear in multiple different orders. OrderItems is used as the bridge

table that solves this problem. This table is also connected to Sellers since different items in a particular order can come from different sellers.

We can insert new records without having any related data existing in the first place, this deals with data anomalies. Updating items like city will only happen in one place. This means that deleting orders won't accidentally delete customer records unintentionally.

On top of this, we put the zip code in orders instead of Customers because for demand forecasting we care about where the orders were shipped and not where the customer actually lives. Also ENUM types on status and payment types keep the data clean. The Zips table has all of the coordinates for heatmaps.

Overall, the final schema meets all the requirements for Third Normal Form. Our structure avoids redundancies and aims to prevent update anomalies. Because of this and normalization, on top of our practical design choices, the schema remains both efficient and aligns to the 3NF principles